

Satellite System Graph: Towards the Efficiency Up-Boundary of Graph-Based Approximate Nearest Neighbor Search

Cong Fu, Changxu Wang, Deng Cai*

The State Key Lab of CAD&CG, College of Computer Science, Zhejiang University, China
Alibaba-Zhejiang University Joint Institute of Frontier Technologies

Fabu Inc., Hangzhou, China

{fc731097343, changxu.mail, dengcai}@gmail.com;

ABSTRACT

Approximate Nearest Neighbor Search (ANNS) in high dimensional space is essential in database and information retrieval. Recently, there has been a surge of interests in exploring efficient graph-based indices for the ANNS problem. Among them, the NSG has resurrected the theory of Monotonic Search Networks (MSNET) and achieved the state-of-the-art performance. However, the performance of the NSG deviates from a potentially optimal position due to the high sparsity of the graph. Specifically, though the average degree of the graph is small, their search algorithm travels a longer way to reach the query. Integrating both factors, the total search complexity (i.e., the number of distance calculations) is not minimized as their wish. In addition, NSG suffers from a high indexing time complexity, which limits the efficiency and the scalability of their method. In this paper, we aim to further mine the potential of the MSNETs. Inspired by the message transfer mechanism of the communication satellite system, we find a new family of MSNETs, namely the Satellite System Graphs (SSG). In particular, while inheriting the superior ANNS properties from the MSNET, we try to ensure the angles between the edges to be no smaller than a given value. Consequently, each node in the graph builds effective connections to its neighborhood omnidirectionally, which ensures an efficient search-routing on the graph like the message transfer among the satellites. We also propose an approximation of the SSG, Navigating SSG, to increase the efficiency of indexing. Both theoretical and extensive experimental analysis are provided to demonstrate the strengths of the proposed approach over the existing state-of-the-art algorithms. Our code has been released on GitHub.

*Corresponding author.

This work is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 12, No. xxx
ISSN 2150-8097.

DOI: <https://doi.org/10.14778/xxxxxxx.xxxxxxx>

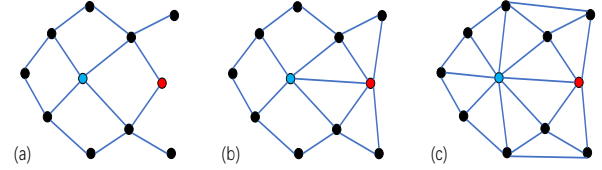


Figure 1: An illustration of how sparsity influences the search complexity of different MSNETs. (a), (b), and (c) are three MSNETs on a toy dataset with different sparsity. In particular, (a) is built according to MRNG’s scheme [15]. The sparsity of the graph decreases from (a) to (c). Take the blue node as the start-search position and the red point as the query. In (a) we need one hop via a neighbor of the blue node to the red one, while in (b) and (c) we only need to check the blue node’s neighbors. As a result, we need 7, 5, and 8 distance calculations to complete the search on (a), (b), and (c) respectively. The most sparse MSNET may not present the best search performance.

PVLDB Reference Format:

Cong Fu, Changxu Wang, Deng Cai. Satellite System Graph: Towards the Efficiency Up-Boundary of Graph-Based Approximate Nearest Neighbor Search. *PVLDB*, 12(xxx): xxx-yyy, 2019. DOI: <https://doi.org/10.14778/xxxxxxx.xxxxxxx>

1. INTRODUCTION

Approximate Nearest Neighbor Search (ANNS) has been a fundamental problem over decades and supports many applications in database, information retrieval, data mining, and machine learning [5, 12, 9, 35, 30, 40, 1]. When deep learning techniques are applied to more and more traditional large-scale applications, indexing and searching on dense-real-vector databases becomes a significant challenge. Due to the intrinsic difficulty of the exact nearest neighbor search, various solutions have been proposed aiming at the ANNS problem. For example, the tree-based methods [7, 16, 36, 23, 13], the hashing-based methods [19, 37, 21, 31], the quantization-based methods [24, 18, 39, 26] and the graph-based methods [2, 20, 32, 34, 6, 15]. Among them, the graph-based methods have shown promising search performance on several widely used public datasets [20, 34, 6, 15]

and been in the leading position. Especially, in the current state-of-the-art work [15], Fu *et al.* resurrects the theory of Monotonic Search Networks (MSNET) [10] and brings it into practice. The MSNET guarantees the search algorithm (the commonly used modified-A*-search algorithm in Alg. 1) to always reduce the distance to the query at each iteration, i.e., the "monotonicity" of the search. Based on the MSNET, Fu *et al.* proposed the Monotonic Relative Neighborhood Graph (MRNG) which is of very low search time complexity (about $O(n^{\frac{1}{d}} \log n)$, where n is the data size and d is the dimension of the data points). They also proposed a practical approximation of the MRNG, namely the Navigating Spreading-out Graph (NSG), with a smaller indexing complexity, which can scale to large-scale problems and is probably the most efficient graph-based ANNS algorithm to the best of our knowledge.

Despite the success of their work, there are mainly two limitations with it (please see Sec. 2 for more details):

- 1) Their indexing scheme produces an over-sparse MSNET, thus leads to an inferior search performance. The search complexity of a graph-based ANNS method is mainly determined by the out-degree and the search path length of the graph (how far Alg. 1 travels to reach the answer). Fig. 1 shows three different MSNETs on a toy dataset, among which (a) is an MRNG built with the edge-selection strategy in the NSG paper [15]. Using Alg. 1, we can reach the query (red node) from the start-search node (blue node) with only one hop on (b) and (c), but two hops on (a). In total, we need 7, 5, 8 distance calculations for (a), (b), (c) respectively to complete the search. Deviated from the understanding of a few previous works [6, 15], the out-degree of the graph is not the smaller the better. There probably exists an "optimal sparsity" for efficient graph-based ANNS.
- 2) The indexing complexity of NSG is still high for large-scale applications (about $O(n^{1.16} + n \log n)$).

In this paper, we aim to address the above two problems and proceed towards the efficiency up-boundary of graph-based ANNS methods, especially for the MSNET family. From our perspective, the search process on a graph index is very similar to the message transferring in a Communication Satellite System. In such a system, the neighbors of each satellite are distributed uniformly around. Consequently, the information propagation is very efficient in any direction. Inspired by this observation, we design a simple but effective edge-selection strategy to inherit the superior ANNS properties from the MSNET family and maintain a uniformly distributed neighborhood simultaneously (by enforcing a minimal-angle-constraint between any two out-edges of each node). Please see Sec. 3 for more details.

The main contributions of our proposal can be summarized as follows:

1. We propose a novel edge-selection strategy and thereby define a new family of MSNETs, the Satellite System Graphs.
2. We provide theoretical analysis for the proposed SSG on its further-developed ANNS properties.
3. We propose a practical variant NSSG to reduce the indexing complexity of the SSG.
4. We perform extensive experiments to demonstrate the significant advantages of the NSSG in both the performance and the scalability. Especially, we test the

Algorithm 1 Search-on-Graph($G, \mathbf{p}, \mathbf{q}, l$)

Require: graph G , start node $\mathbf{p}$, query point $\mathbf{q}$, candidate pool size l

Ensure: k nearest neighbors of $\mathbf{q}$

```

1:  $i=0$ , candidate pool  $S = \emptyset$ 
2:  $S.add(\mathbf{p})$ 
3: while  $i < l$  do
4:    $i$  = the index of the first unchecked node in  $S$ 
5:   mark  $\mathbf{p}_i$  as checked
6:   for all neighbor  $\mathbf{n}$  of  $\mathbf{p}_i$  in  $G$  do
7:     if  $\mathbf{n}$  has not been compared then
8:        $S.add(\mathbf{n})$ 
9:     end if
10:  end for
11:  sort  $S$  in ascending order of the distance to  $\mathbf{q}$ 
12:  if  $S.size() > l$  then
13:     $S.resize(l)$ 
14:  end if
15: end while
16: return the first  $k$  nodes in  $S$ 

```

NSSG on a 100-million-point dataset on a single CPU to show its advantages on very large datasets.

2. BACKGROUND

2.1 From NNS to ANNS

Nearest Neighbor Search (NNS) is a fundamental problem in database, data mining, and information retrieval community. It also supports various applications of machine learning. Nearest neighbor search algorithms return the nearest data points of a given query point in a certain metric space. Exact nearest neighbor search is impractical in large-scale applications due to its high time cost. Thus, people turn to ANNS techniques. The ANNS methods all try to index the datasets and perform non-exhausted search based on such indices. The non-exhausted search algorithms only scan a part of points in the database and get the answers with a little accuracy loss efficiently, *i.e.*, sacrificing accuracy for efficiency. In real-world applications, the approximate K nearest neighbor search (AKNNS, sub-problem of the ANNS problem) is more commonly used. Please refer to [15] for formal definitions.

2.2 Non-Graph Based Methods

For the past decades, various methods are proposed to solve the AKNNS problem efficiently, including the hashing-based, the tree-based, the quantization-based, and the graph-based methods. The hash-ing-based methods try to split the space with hyper-surfaces and organize the dataset with hashing tables, typical methods include Locality Sensitive Hashing (LSH) [19] and Spectral Hashing [37]. The tree-based methods try to partition the space into sub-regions and index them into tree-structures. Representative methods include Randomized KD-Tree [36] and R-Tree [4]. The quantization-based methods try to solve the AKNNS problem through reducing the complexity of distance calculations. Specifically, the algorithm quantizes the original data points and represents them as binary codes, which serve as the references to the codebook (quantizers). The complexity of the distance computation can be reduced significantly by computing the approximate distance with the pre-built codebook (quantizers). Product Quantization (PQ) [24] and Composite Quantization[39] are two typical methods. There

are some recent works focusing on improving the performance of these three types methods in the methodology or engineering level such as [22, 1, 17].

2.3 Graph-Based Methods.

Recently, the graph-based methods have attracted wide interests and shown exciting results. Although various graph indices have been developed, they all use the same search algorithm (shown in Alg.1). The main idea of this A*-search-like algorithm is to iteratively discover the nodes which are closer to the query, among the neighbors of the current node. "Neighbors of neighbors are also likely to be neighbors". Thus, no matter how complex the graph is, the search time complexity of them can be computed in the decomposition of ol [15], where o is the out-degree of the graph and l is the number of iterations of Alg.1 (or the length of the "search path"). From this perspective, the development of the graph indices can be summarized as two aspects: sparsify the graph and reduce the search path lengths.

There are several early graph models with excellent theoretical guarantees (e.g., the Delaunay Graphs [3, 28] and the MSNET [10]) or empirical conclusions ([27, 8]) on the search path length. However, these graphs suffer from high indexing complexity or are not well-designed models for the ANNS problem. Recent works are mostly optimized approximations of above structures. Specifically, KNN graph based methods (GNNS [20], IEH [25], Efanna [14]) stem from the Delaunay Graph. They are designed to reduce the out-degree of the Delaunay Graph in the high dimensions and maintain a considerably short search path; Navigating Small World Graph (NSW) [32] approximates the Navigating Small World Network [27] and is modified to adapt to high dimensions. Hierarchical Navigating Small World Graph (HNSW) [34] further improves the NSW by stacking multiple NSWs of different scopes, which intuitively shortens the search paths via different short-cuts on different graph layers; FANNG [6] and HNSW [34] all use similar edge-selection strategies as the Relative Neighborhood Graph (RNG) to sparsify the graphs; Based on the MSNET [10], Fu *et al.* develop the MRNG [15] with a good out-degree upper bound and a short search path guarantee. Further, they propose the NSG to approximate the MRNG to reduce the indexing complexity. In the extensive experimental study of several graph-based papers [20, 25, 14, 33, 34, 6, 15], the graph-based approaches outperform the non-graph based ones significantly on several frequently-used public datasets. Among them, NSG [15] is in the leading position to the best of our knowledge.

2.4 Closely Related Works

MSNET and MRNG. MSNET [10] is proved to guarantee that the search algorithm (Alg.1) will always find a better node from the neighbors of the current node. The "better node" is some node which must be closer to the query node than the current node. This property is called the monotonicity. Fu *et al.* [15] prove that the search complexity on an MSNET is $O(n^{\frac{1}{d}} \log n / \Delta r)$, where Δr can be treated as a constant in practice. They further propose the MRNG, which inherits the MSNET's properties with a low out-degree.

It is important to note that MSNETs cover a large family of different graphs. Although the search time complexity of different MSNETs grows with n in the same speed, the

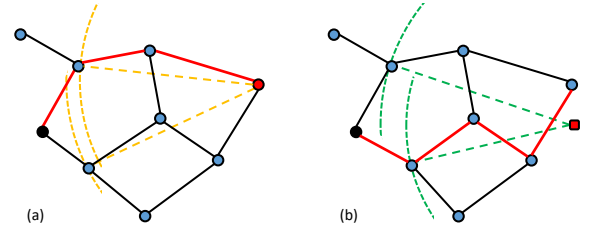


Figure 2: An illustration of the difference between searching for an in-database query and a not-in-database query. The black node denotes the search-start node, and the red nodes denotes the query nodes. (a) describes the search for an in-database query, while (b) describes that for a not-in-database query. The distance to the query determines the choice of the first step. Therefore, though they arrive at the same location finally, the routes are completely different (red lines).

constant factors vary with different graph structures. For example, the Delaunay Graph is an MSNET. Though the search paths on a Delaunay Graph are usually very short, but the out-degree of the graph is too high in a high dimensional space [10, 6]. Searching on an exact Delaunay Graph may be as slow as the exhausted search. On the contrary, the principle of the MRNG [15] is to prune the graph as much as possible, as long as it is still an MSNET. In an MRNG, the longest edge in a triangle must be removed, thus, the average angle between edges will be much larger than 60° , which leads to a high sparsity. The fact is that, as shown in Fig.1, a sparser graph does not lead to a better search performance. Removing an edge can reduce the degree, whereas there is also a probability to corrupt a short path, and consequently the search algorithm needs to find a longer alternative way. Based on our experimental study, we believe there exists an optimal average out-degree (sparsity) for the MSNET-based search. From this perspective, the disadvantage of the MRNG is that their indexing algorithm cannot adjust its degree flexibly.

DPG. Similar to our work, the Diversified Proximity Graph (DPG) [29] uses angles between out-edges to filter neighbors for each node. However, there are three differences between the DPG and our work SSG: 1) SSGs belong to the MSNET family, while the DPG is not; 2) We enforce the angles between out-edges to be no smaller than a fixed value, while the DPG presets the number of reserved out-edges and preserves those edges which maximize the average angle between the edges. 3) An SSG is a directed graph, while the DPG is undirected.

IN- and NOT-IN-DataBase queries. In real applications, we usually deal with two different types of queries. For example, sometimes we are required to retrieve the clothes in a certain online shop (in-database query). Sometimes we are required to find some similar clothes according to the descriptions from the customer (not-in-database query). When searching for in-database queries, we can retrieve both the queries themselves and their neighbors from the database ideally. When searching for not-in-database queries, we are only required to return the nearest neighbors of the queries. The intrinsic dynamics of the graph-based methods is the

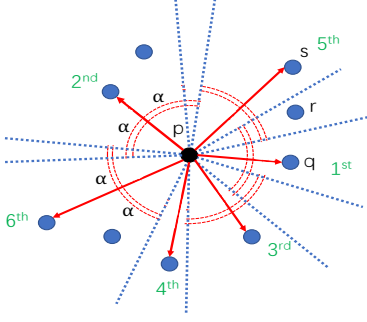


Figure 3: An illustration of selecting edges for one point in a toy 2D setting. The red directed edges are the selected edges in the toy SSG for the black point. The edges are selected in the order of edge length, and the green numbers are the order in which they are selected. For example, according to the SSG definition, $\vec{pq} \in SSG$ because $Cone(\vec{pq}, \alpha) \cap S = \emptyset$. $\vec{ps} \in SSG$ because $r \in S \cap Cone(\vec{ps}, \alpha)$ but $\vec{pr} \notin SSG$.

greedy routing, which differentiates the search of in-database queries from the not-in-database queries significantly. Fig. 2 shows the differences for different queries. For the in-database and not-in-database queries, the ground-truth is the same node (the rightmost node). However, Alg. 1 will choose completely different paths to reach it. Intuitively, searching for not-in-database queries needs more steps than for in-database queries. This phenomenon has never been discussed in previous works. The theoretical analysis in Fu *et al.*'s work [15] only focuses on in-database queries.

In the coming section, we will present theoretical analysis of the SSGs in terms of both aspects.

3. SATELLITE SYSTEM GRAPH

As we discussed in Section 1 and 2, the current state-of-the-art graph-based approach NSG [15] has the following two problems: 1) the graph is over-sparse, which damages the search efficiency; 2) the high indexing complexity limits its scalability and efficiency; Problem 1) is related to the graph's theoretical definition while problem 2) is related to the indexing algorithm design.

For problem 1), we will introduce a new family of MSNETs with better theoretical properties in Sec. 3.2, namely the SSGs. All SSGs belong to the MSNET family, and we can use a parameter (describing the minimal angle between out-edges) to control the sparsity of the graph, which enables us to explore the optimal out-degree. For problem 2), we design a novel indexing algorithm to approximate the SSG (building exact SSGs is time-consuming) in Sec. 4. The resulting indexing algorithm is of low time complexity, which benefits large applications.

3.1 SSG and Naive Indexing

Preliminaries. Given a node set S , a graph G is typically defined as a set of edges linking the nodes in S . Let n denote the number of nodes in S , and d denotes the dimension of the corresponding data space. The graph-based ANNS is to reach the query q from a start-search node p with Alg. 1. Further, a graph is "monotonic" if in each iteration of the



Figure 4: An comparison between the MRNG and the SSG. (a) is a toy MRNG, (b) is a toy SSG. In the MRNG, the longest edges in a triangle must be removed. Thus, there will be vacant areas without out-going edges like the dotted lines in (a). In the SSG, we filter the out-going edges by the angles between them, which is more flexible and can reduce the vacant areas with a proper angle chosen.

search process, Alg. 1 goes to a position which is closer and closer to q . All monotonic graphs belong to the MSNET family [10]. For the convenience of analysis, we formally define the **Satellite System Graphs** as follows.

Definition 1 (SSG). In Euclidean space, for an edge $\vec{pq}$, $Cone(\vec{pq}, \alpha)$ is defined as the circular cone centered at $\vec{pq}$ whose angular diameter is 2α . An SSG is defined as the set of directed edges satisfying the property: for any edge $\vec{pq}$, $\vec{pq} \in SSG$ if and only if $Cone(\vec{pq}, \alpha) \cap B(p, \delta(p, q)) \cap S = \emptyset$ or $\forall r \in Cone(\vec{pq}, \alpha) \cap B(p, \delta(p, q)) \cap S, \vec{pr} \notin SSG$, where $B(p, \delta(p, q))$ is defined as the open sphere centered at p with radius $\delta(p, q)$, and $\delta(p, q)$ is the distance between p and q . $\alpha \leq 60^\circ$.

Naive SSG Indexing Routine. Building an SSG is quite simple. Given a finite node set S , we perform the following steps for each point $p \in S$ to obtain an SSG (Figure 3 is an illustration of these steps on a toy dataset):

1. For each point $q \in S - \{p\}$, calculate the length of pq .
2. Sort the set $q \in S - \{p\}$ in the ascending order of the length $\vec{pq}$, which forms the list C .
3. For each edge in $\vec{pq} \in C$, check whether $\vec{pq}$ can be added to SSG in order according to the SSG definition.

3.2 ANNS Properties of SSG

In this paper, we go beyond the previous researches and for the first time analyze the ANNS properties of the SSG theoretically in two scenarios: **in-database-query** and **not-in-database-query**.

3.2.1 In-Database Search.

SSGs belong to the MSNET family, thus, they share the same properties for the in-database queries with other MSNETs:

Theorem 1. SSG is monotonic and strongly connected.

Theorem 2. For a finite point set S randomly distributed in the Euclidean Space E^d and any query $q \in S$, the search complexity from a random starting point is $O(Dn^{1/d} \log(n^{1/d})/\Delta r)$, where D is the degree upper-bound of SSG, n is the size of the dataset, and Δr is the lower-bound of length differences of edges in any non-isosceles triangles.

Algorithm 2 SSGIndexing(D, l, r, s, α)

Require: dataset D , candidate set size l , maximum out-degree r , number of navigating points s , minimum angle α .

Ensure: an SSG G .

```
1: Build an approximate  $k$ NN graph  $G_{knn}$ .
2:  $G = \emptyset$ .
3: for all node  $i$  in  $G_{knn}$  do
4:    $P = \emptyset$ .
5:   for all neighbor  $n$  of node  $i$  do
6:      $P.add(n)$ .
7:     for all neighbor  $n'$  of node  $n$  do
8:        $P.add(n')$ .
9:     end for
10:    remove the redundant nodes in  $P$ .
11:    if  $P.size() \geq l$  then
12:      break.
13:    end if
14:  end for
15:  sort  $P$  in the ascending order of the distance to  $i$ .
16:  select neighbors from  $P$  according to the definition
    of SSG.
17: end for
18: Random select  $s$  points from the datasets as  $N$ .
19: for all point  $n$  in  $N$  do
20:   Strengthen the connectivity of the graph with
21:   DFS-expanding from  $s$ .
22: end for
23: return  $G$ .
```

Please see the proofs in the Appendix.

The search complexity of the MRNG [15] and the SSG seems the same, but the constant factors (e.g., the degree D) will influence the actual value, which is difficult to calculate. We conduct experimental study on the MRNG and the SSGs (shown in the experiment section later). The results show that the average out-degree of the MRNG is lower than the SSG, whereas the search path lengths of the SSG are shorter than the MRNG. This agrees with our expectation. The reason is straightforward. Considering the structure of the satellite system, the key to efficient message transfer is to fill the neighborhood of each satellite with evenly distributed neighbors. As shown in Fig. 4, candidate neighbors of the central node are highly mutually excluded in an MRNG. As a result, many "vacant" areas in the neighborhood are created, which causes high sparsity and the "detouring search" in Fig. 1. These "vacant areas" can be filled with effective edges when a proper α is chosen in the SSG. Though the search path may be shortened, the out-degree of the graph may be enlarged accordingly. Intuitively, smaller α will allow more edges to be added into the graph.

To approach the up-boundary of the graph-based search, we should carefully select effective edges to fill up the "vacant areas". This is our main motivation to design the SSG. With the angle constraint α , we can adjust the graph to adapt to different data distribution and find a trade-off of the search path length and the out-degree. Whereas in the NSG paper [15], the MRNG is defined to be deterministic. In other words, given a fixed node set, only one MRNG can be built out of it. The MRNG cannot enrich itself with more effective edges flexibly.

3.2.2 Not-In-Database Search.

Theorem 3. *For a finite point set S randomly distributed in the Euclidean Space E^d and any query $q \notin S$, the probability that each step on the search path is monotonic to both r and q is $0.5 + \epsilon, 0 < \epsilon \leq 0.5$, under the condition that for any neighbor s of the node p in the current search step, $\delta(p, s) < \delta(p, q)$. Further, $\epsilon = 1$ when we set $\alpha \leq 30^\circ$ for the SSG.*

Please see the proof in the Appendix.

We can see that another advantage of the SSG over the MRNG is related to the property for not-in-database queries, which comes out of the angle constraint of the SSGs as analyzed in the proof. For queries not in the database ($q \notin S$), different graphs in the SSG family behave differently. For the SSG with $\alpha = 60^\circ$, there is a probability of less than 0.5 for the search algorithm to violate the monotonicity at each step, whereas for the SSG with $\alpha \leq 30^\circ$, the graph is guaranteed to be monotonic for both in-database and not-in-database queries. Meanwhile, we should notice that SSG($\alpha = 60^\circ$) is much more sparse than SSG($\alpha \leq 30^\circ$). For not-in-database queries, there is a trade-off between the monotonicity and the out-degree of the graph. The best α should also be determined by the data distribution. MRNG does not have angle-constraints on the graph, thus, the above property is not tenable on the MRNG, nor the other previous works.

In summary, SSGs define a family of MSNETs with a tunable parameter to control the sparsity. One can explore the best graph structure among them for different data distributions. Meanwhile, SSGs have better theoretical generalizability than the MRNG on not-in-database queries.

4. A PRACTICAL VARIANT : NSSG

Although SSG has very good properties for ANNS problems, it is impractical to build an SSG for large-scale datasets. Because it is easy to verify that the time complexity of the naive SSG construction algorithm in Sec 3 is $O(n^2 \log n)$.

In this section, we propose an approximation of SSG to reduce the indexing time complexity and maintain the excellent ANNS properties approximately, which is named as the Navigating SSG (NSSG). The main idea is 1) selecting neighbors from a small effective candidate set instead of from all the nodes; 2) applying the SSG's edge-selection strategy on the candidates; 3) maintain the connectivity of the graph (making sure each node is reachable).

Retrench the candidates. In our experimental study (shown in latter section), a small local neighborhood covers enough effective neighbors for each node in terms of search routing (like the satellite system, in which each satellite does not have connections with those very distant satellites). Adding long edges provides limited contributions in terms of shortening the search paths. The edges in the SSG are selected from all edges in the complete graph on S , which is time- and space-consuming. To reduce the indexing complexity, we just collect a small set of effective neighbors for each node, i.e., we focus on each node's close neighborhood to select neighbors. To achieve this objective, we first build a KNN graph with a small $k, k \ll n$. Then we expand the candidate set by collecting the neighbors of the current node and its neighbors' neighbors, which is much faster than the search-and-collect approach of the NSG [15].

Effective edge selection. Because effective edges crowd in a small neighborhood, the key to approximate the SSG is to inherit the structures in the local neighborhood of each node. Therefore, we require a high precision of the KNN graph (usually above 90% in practice) and just apply the SSG edge-selection strategy on the candidate set collected in the above way, to copy the local structures of the SSG approximately. We also set a max-degree limitation for the final graph to avoid the "degree explosion" for few points.

Maintain the connectivity. Follow the idea of the NSG [15], we guarantee the connectivity of the graph by fixing the start-search node. It is difficult to make the whole approximate graph strongly-connected with minimal edges added, but it is easy to ensure the connectivity from one node to all the others. Fu *et al.* [15] propose a DFS-expanding algorithm to add a small number of edges to the graph to ensure this single-direction connectivity. The main idea of the DFS-expanding algorithm is to generate a DFS tree from a selected node and try to connect all the other nodes to the tree (please refer to [15] for more details). This selected node is called the navigating node, from which we initiate the search algorithm. Instead of selecting the central node as the only navigating node (what NSG does), we random select m navigating nodes ($m \ll n$, e.g., we use $m = 10$ in our experiments) and strengthen the connectivity upon all of them with the DFS-expanding algorithm. Due to the random selection, these navigating nodes are expected to be distributed in the dataset uniformly. Compared with the NSG (starting all the search from the "central-hub"), the NSSG can deal with data with different shapes or distributions better.

Although there is an constraint on α on the SSGs ($0^\circ < \alpha \leq 60^\circ$), we are actually free to choose all α in the range $0^\circ < \alpha \leq 90^\circ$. For the NSSGs with $60^\circ < \alpha < 90^\circ$, they are no longer the approximations of the SSGs. Such NSSGs can also be used for the ANNS problem, but the performance is much worse (shown in the experiment section later). The indexing pseudo-codes of the NSSG is given in Algorithm 2.

As for the search algorithm on the NSSG, we use Alg. 1, the same as all the other graph-based methods. A little modification is that we first compare all the navigating nodes to the query and start the search from the one which is the nearest to the query. Our code has been released on GitHub¹.

4.1 Complexity Analysis

The indexing process of the NSSG contains three parts: 1) collecting candidates, 2) selecting edges, and 3) strengthening the connectivity.

For part 1), we build an approximate KNN graph with a small K using the nn -descent algorithm [11] and expand the neighborhood by retrieving the neighbors of neighbors. The complexity of building the approximate KNN graph is $n^{1.16}$ [11]. For neighbor expansion on the KNN graph, the expansion only needs the table-look-up operation. The complexity is $O(nK^2)$. Thus, the total complexity of Part 1) in Algorithm 2 is $O(nK^2 + n^{1.16})$.

The complexity of edge selection is $O(n(l \log l + r^2)d)$, where l is the size of the candidate set, r is the max-degree, and d is the dimension.

¹<https://github.com/ZJULearning/SSG>

Table 1: Information of experimental datasets. We list the dimension (D), the local intrinsic dimension (LID), the number of base vectors, and the number of query vectors.

Dataset	D	LID	No. of base	No. of query
SIFT10K	128	9.9	10,000	100
SIFT1M	128	12.9	1,000,000	10,000
GIST1M	960	29.1	1,000,000	1,000
Crawl	300	15.7	1,989,995	10,000
GloVe	100	20.9	1,183,514	10,000
Deep100M	96	10.2	100,000,000	10,000

The complexity of Part 3) is quite small and grows linearly with n according to [15].

The search complexity of NSSG is hard to calculate because we do not know when the iteration ends. Our empirical study shows that the search complexity on the NSSG is close to $O(\log n)$ and is lower than the NSG's.

5. EXPERIMENTS AND ANALYSIS

In this section, we will present the experimental results and detailed analysis to demonstrate the effectiveness of our method.

5.1 Datasets

We use the sift10K² dataset to evaluate the MRNG and the SSG. We use five different public real-world datasets to evaluate the performance of different algorithms, including four million-scale datasets, *e.g.*, SIFT1M³, GIST1M⁴, Crawl⁵, and GloVe⁶, and one large-scale dataset, Deep100M. The Deep100M is sampled from the Deep1B dataset⁷. The datasets come from different media (*i.e.*, text and image). The intrinsic dimensions of these datasets are quite different. We list the details of these datasets in Table 1.

For those algorithms that cannot scale to the dataset with 100 million points, such as HNSW, KGraph, Annoy, and FLANN, we just run them on the medium-size datasets with single thread. For NSSG, NSG, and Faiss, we evaluate them on both medium-scale and large-scale datasets. Meanwhile, we evaluate the inner-query parallelism of NSSG, NSG, and Faiss on Deep100M with 16 threads.

5.2 SSG v.s. MRNG

Due to the high indexing complexity, we only evaluate the performance of the SSGs and MRNG on the SIFT10K dataset. For the SSG family, we select two typical graphs, the SSG_{60°} and the SSG_{30°}. The SSG_{60°} is the most sparse graph in the SSG family according to the definition ($0^\circ < \alpha \leq 60^\circ$). As proved above, the MRNG and the SSGs with $30^\circ < \alpha \leq 60^\circ$ are only monotonic for the in-database queries. The SSGs with $0 < \alpha \leq 30^\circ$ are monotonic on both in-database and not-in-database queries. Among them, SSG_{30°} is the most sparse one.

²<http://corpus-textmex.irisa.fr/>

³<http://corpus-textmex.irisa.fr/>

⁴<http://corpus-textmex.irisa.fr/>

⁵<http://commoncrawl.org/>

⁶<https://nlp.stanford.edu/projects/glove/>

⁷<https://github.com/facebookresearch/faiss/tree/master/benchs#getting-deep1b>

Table 2: The results of the SSGs and the MRNG on SIFT10K. AOD denotes the average out-degree of the graph, MOD denotes the max-out-degree of the graph, L_{in-DB} denotes the average search path length for in-database queries, and $L_{not-in-DB}$ denotes the average search path length for not-in-database queries.

Graph	AOD	MOD	L_{in-DB}	$L_{not-in-DB}$
MRNG	18	66	2.76	4.98
SSG _{60°}	40	111	2.18	3.91
SSG _{30°}	121	746	1.80	1.79

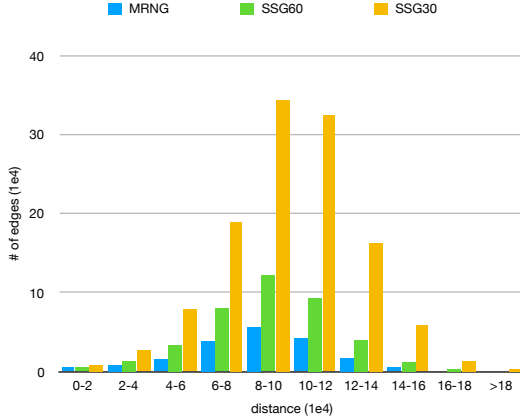


Figure 5: The edge-length distributions of the exact MRNG, SSG_{60°}, and SSG_{30°}.

From Table 2, the statistics meet our expectation. 1) The MRNG is much more sparse than the SSGs, especially more sparse than the most sparse SSG (SSG_{60°}). 2) The average lengths of the search paths for in-database queries are consistently shorter than not-in-database queries. 3) The average lengths of the search paths on SSGs are shorter than on the MRNG. The denser the graph is, the shorter the paths are.

It is important to notice that SSGs have no advantages on the out-degree of the graph, though the search paths are shorter. To dig deeper into the graph structure, we show the length distribution of all these graphs in Fig. 5. We can see that the major part of edges in the MRNG are shorter than 120000, which indicates that a sphere of radius 120000 is an effective neighborhood coverage for search routing. From SSG_{60°} to SSG_{30°}, a large number of edges are added in the range 60000 – 160000. From the MRNG to the SSG_{60°}, the edges are more than doubled, but the average length of the search paths is not reduced by a half. From the SSG_{60°} to the SSG_{30°}, the edges are doubled again, but the contribution in shortening the search paths is even less.

In summary, the edge-selection strategy of SSG can locate effective edges for shortening the search path. The side effect is that it also brings in a large number of ineffective long edges. This finding is why we only focus in the small neighborhood of each node and set a max-degree limitation while building the NSSG (discussed in Sec. 4).

5.3 NSSG v.s. SOTA methods

Table 3: Information of the graph-based indices involved in all of our experiments. The Size means the memory occupation of the index. AOD means the Average Out-Degree. MOD means the Maximum Out-Degree. Because HNSW contains multiple graphs, we only report the AOD and MOD of its bottom-layer graph here.

Dataset	Algorithms	Size (MB)	AOD	MOD
SIFT1M	NSSG	153	39	50
	NSG	153	25.9	50
	HNSW	451	32.1	50
	KGraph	374	200	200
	DPG	632	165.1	1260
GIST1M	NSSG	267	34	70
	NSG	267	26.3	70
	HNSW	667	23.9	70
	KGraph	1526	400	400
	DPG	741	194.3	20899
Crawl	NSSG	303	22	40
	NSG	303	11	40
	HNSW	759	12.1	40
	KGraph	3036	400	400
	DPG	1465	193	97189
GloVe	NSSG	225	21	50
	NSG	225	13	50
	HNSW	564	12	50
	KGraph	1805	400	400
	DPG	787	174	50336

Table 4: The indexing time of all the graph-based methods. The indexing times of NSSG and NSG are recorded in the form $t_1 + t_2$, where t_1 is the time to build the approximate KNN graph, and t_2 is the time of edge-selection and connectivity strengthening.

Dataset	NSSG	NSG	HNSW	KGraph	DPG
SIFT1M	62+13	62+45	149	62	1120
GIST1M	620+144	620+735	1376	620	6700
Crawl	790+82	790+567	1083	790	9169
GloVe	650+18	650+102	930	650	3139

5.3.1 Evaluation.

We evaluate the search performance of the compared algorithms by queries-per-second v.s. precision, *i.e.*, we record how much queries an algorithm can process per second at given precision. The precision is formally defined as:

$$precision(R) = \frac{|R \cap G|}{|G|} \quad (1)$$

where R is the answer set returned by the algorithm, and G is the ground-truth set of the given query [15].

In practical scenario, we are always concerned more about the search time in high-precision area. Because with different hyper-parameters each algorithm will perform very differently in different precision area, we will try to present the best performance of all the algorithms in the high-precision region via parameter-tuning. In order not to lose generality, we divide each dataset into training set, validation set, and test set. We tune the parameters on the validation set

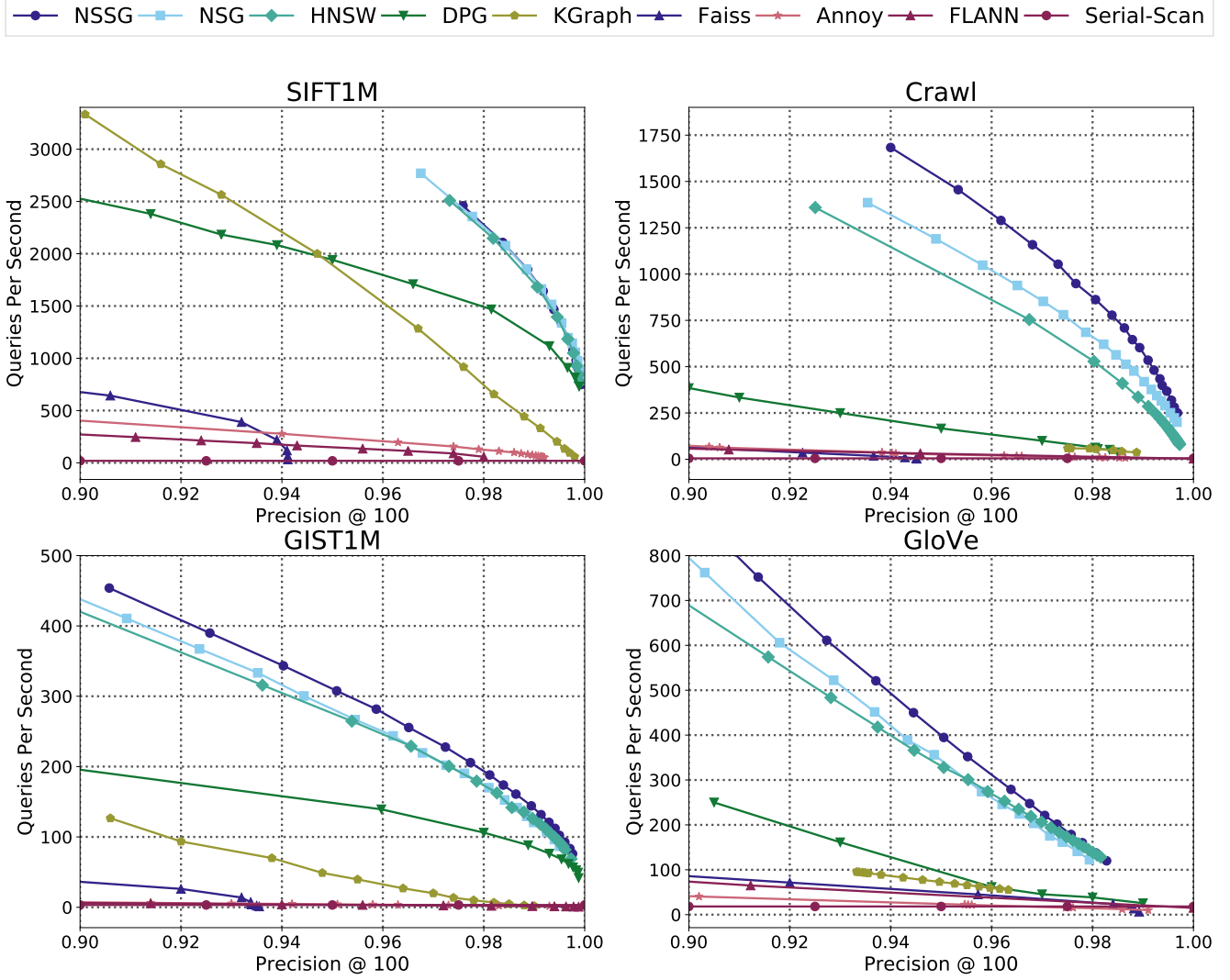


Figure 6: ANNS performance of graph-based algorithms with their optimal indices in high-precision region on the four datasets (top right is better). The x-axis is not meaningful for Serial-Scan because the results are accurate.

to get the best-performing index of each algorithm at high-precision region.

Moreover, we record the indexing time, memory usage of the indices, and other useful index information for the graph-based algorithms.

5.3.2 Compared Algorithms.

We select seven state-of-the-art algorithms from different types. We do not compare with the hashing-based methods because they are generally too slow at high-precision region (much slower than the serial-scan). All the algorithms are run on a machine with an Intel i9-7980XE CPU and 128 GB memory. The indices are built with all 36 threads but the search is evaluated on a single thread. Specifically, the selected algorithms are listed as follows:

1. **FLANN**⁸ is a well-known ANNS library based on

⁸<https://github.com/mariusmuja/flann>

many tree-based algorithms, including randomized KD-trees, Kmeans trees, and so on. We use its auto-tune composite tree algorithm for comparison.

2. **Annoy**⁹ is a K-means tree algorithm with $K = 2$. The algorithm is specially optimized for $K = 2$.
3. **Faiss**¹⁰ is a quantization-based algorithm recently released by Facebook. We use its IVF-PQ implementation for comparison. Specifically, the index contains two parts: the inverted file (IVF) and the product quantization code and the codebook. The search on the IVFPQ is a two-stage process: use the IVF to locate a small number of candidate answers and then use the quantized distance to rank them.

⁹<https://github.com/spotify/annoy>

¹⁰<https://github.com/facebookresearch/faiss>

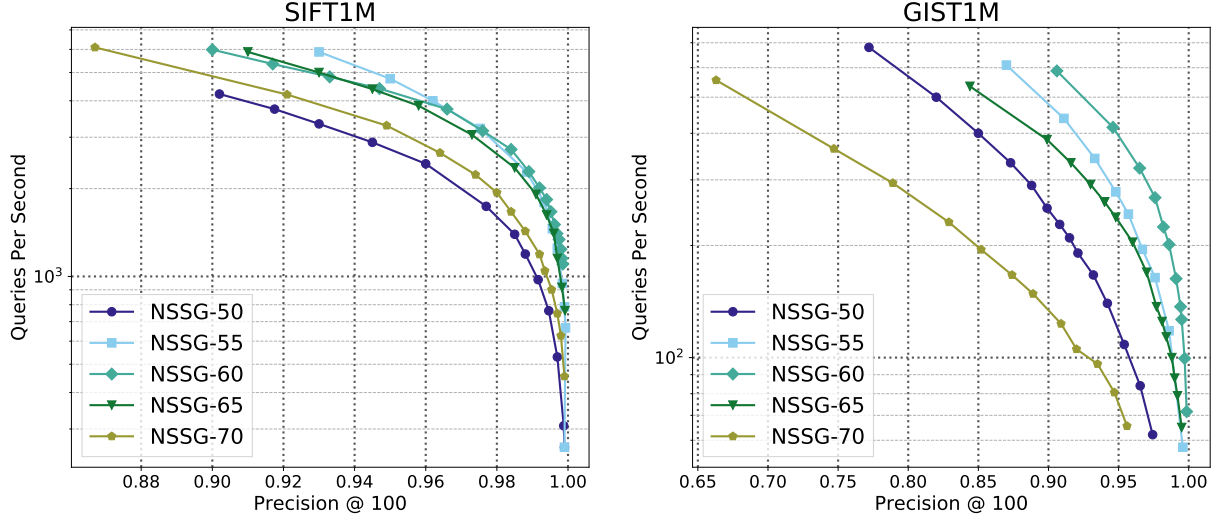


Figure 7: The results of using different angles on GIST1M datasets. The performance with 60° is much better than the other angles.

4. **KGraph**¹¹ is a graph-based method which use a KNN graph as the index. It is a well-implemented version of the GNNS [20] algorithm. It also contains a well-implemented *nn*-descent algorithm [11] to build the KNN graph.
5. **DPG**¹² is a graph-based method which also uses the angles between the edges to select edges [29]. The differences between the DPG and the SSG are discussed in Sec. 2.
6. **HNSW**¹³ is a well-known graph-based algorithm based on a structure named as Hierarchical NSW Graph [34].
7. **NSG**¹⁴ is the current state-of-the-art graph-based algorithm which is the approximation of the MRNG [15].
8. **Serial-Scan**. We perform serial-scan on the test sets to demonstrate the speed-up of different algorithms. Note that the x-coordinate in Figure 6 is meaningless to Serial-Scan, since the Serial-Scan always returns the ground truth.
9. **NSSG** uses Algorithm 2 to build the NSSG. The KNN graph construction algorithm we use here is the *nn*-descent algorithm [11].

6. RESULTS AND ANALYSIS

6.1 Experiments on Medium Datasets

We first present the results on the four medium-size datasets as follows.

¹¹<https://github.com/aaalgo/kgraph>

¹²https://github.com/DBWangGroupUNSW/nns_benchmark

¹³<https://github.com/searchivarius/nmslib>

¹⁴<https://github.com/ZJULearning/nsq>

6.1.1 Search Performance.

The search performance on the four medium-scale datasets are shown in Figure 6. We can see that:

1. Our approach outperforms the others significantly. Especially on datasets with higher intrinsic dimension (GIST1M, Crawl, and GloVe), the gap between our method and the others becomes wider and wider. This indicates our method is insensitive to the increase of the dimension compared with the others.
2. Graph-based methods are much better than non-graph-based ones on these datasets, which is consistent with the results in the experimental report of other works[20, 25, 6, 15, 34].
3. Both using the angle as the edge filter, the DPG performs far worse than the NSSG, especially on higher dimensions. The main reason is that the DPG is not an MSNET. Thus, there is no theoretical guarantee for the performance. In addition, the DPG is too dense (Table 3). There are too much ineffective edges in the DPG.
4. The performance of the NSSG is much better than the NSG and the HNSW, even though the NSSG is denser than them. This is because the edge-selection strategy of the NSSG brings in effective edges for search routing, and searching on the NSSG needs much less hops to reach the desired answers. Meanwhile, we limit the out-degree of the NSSG to a moderate level. Overall the search complexity of the NSSG is smaller.

6.1.2 Indexing Performance.

We lists the details of indices of various algorithms in Table 3 and 4. The index size of the graph is determined by the max-degree of the graph, because all the graph-based methods align the neighbor adjacent list to the same size

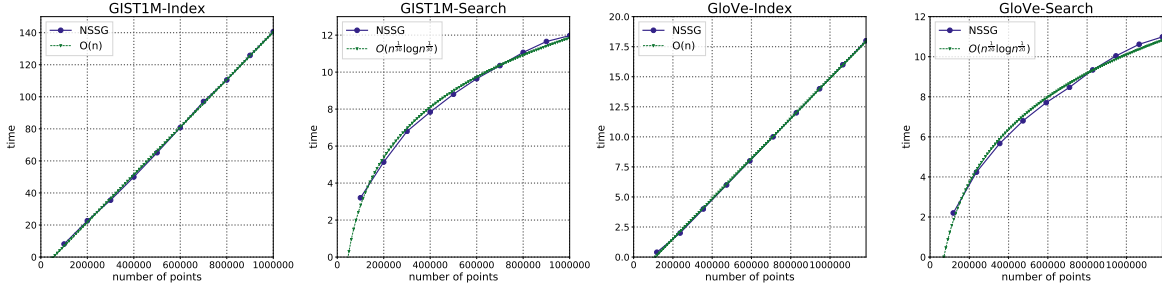


Figure 8: The results of the indexing time complexity and searching time complexity experiments on the GIST1M dataset.

for continuous memory access. This operation is not suitable for the DPG because its max-out-degree is too large. In order to fit in the memory, the DPG has to use the unaligned vectors. We can see that:

1. NSSG and NSG’s indices are the smallest among all the graph-based methods because their optimal indices have a small max-degree. Though the max-degree of the HNSW is the same with the NSSG and the NSG, the index size of the HNSW is large because it contains multiple layers of graphs in its index.
2. The indexing time of the NSSG is the second fastest among all the graph-based methods. The KGraph is faster than NSSG because the KGraph (the nn -descent algorithm) is the first step of the NSSG indexing algorithm (Alg. 2). However, NSSG is much more efficient than the KGraph in the search performance.
3. We notice that the KNN graph construction part is the most time-consuming part in Alg. 2. In practice, this part can be easily replaced with other more efficient algorithm or implemented on other platforms such as GPU to extend the scalability.

6.1.3 Parameters

To build the NSSG index, there are several parameters tunable: the size of the candidate-neighbor-pool l , the max-out-degree of the graph r , the number of navigating nodes m , the minimal angle between edges α .

We test multiple different angles on different datasets, interestingly, we find 60° is the best on all these four datasets. Thus, we set $\alpha = 60^\circ$ for all these datasets. Although the NSSGs with $\alpha > 60^\circ$ are not the approximations of the SSGs as discussed in Sec. 4, we still show their performance here to demonstrate the trend. Due to the limitation of the space, we only put the angle-test results of SIFT1M and GIST1M here (Figure 7).

For the remaining indexing parameters of NSSG and the indexing parameters of other algorithms, we search for the optimal parameters by scanning the enumerations of their parameters on the validation set (the validation set is a 1% subset random sampled from the corresponding base set). We list the optimal values of important parameters for different algorithms and datasets in the Appendix. As for the meaning of the parameters of the other algorithms, please refer to their codes on GitHub.

6.1.4 Complexity

We split the medium-scale datasets into several subsets with different sizes, build the NSSG indices on these datasets, and record the searching and indexing performance statistics. To evaluate the indexing complexity, we use the same parameters to construct indices on all the subsets and record the time. It is important to note that the indexing time we report is only the time after the KNN graph is built. We do not include the KNN graph construction part in this indexing complexity experiments because the complexity of that part is not related to our work.

To evaluate the searching complexity, we use the indices built above to conduct search on each subset and share the same query set. We record the search time for the results whose precision is about 99.75%. Due to the space limitation, we only show the search and indexing complexity experiments on the GIST1M dataset in Figure 8. The behaviors of the NSSG on other datasets follow similar patterns.

The indexing complexity of the NSSG contains three parts: the KNN graph construction (we do not discuss here), the candidate generation and the edge-selection. For the candidate generation and edge selection part, it is linear and much lower than the NSG as shown in Fig. 8, which agrees with our analysis in previous section.

Due to the space limitation, we only show the search complexity results on GIST1M here. The search complexity of our approach is about $O(n^{\frac{1}{18}} \log n^{\frac{1}{18}})$, where 18 is close to the intrinsic dimension of GIST1M. The curve is also very close to $O(\log n)$. On the other datasets, NSSG behaves in a similar pattern. Therefore, our estimation of the search complexity of NSSG is $O(n^{\frac{1}{d}} \log n^{\frac{1}{d}})$, where d is the intrinsic dimension. The behaviors of the NSSG on other datasets follow similar patterns. This agrees with our theoretical analysis. The search complexity of the NSSG scales with n in the similar speed as the NSG does. But we can see it is lower than the NSG’s due to the constant factors. We can conclude that searching on NSSG is more efficient than with the NSG.

6.2 Performance on Deep100M

We only evaluate the performance of the NSSG, NSG, and Faiss on the Deep100M datasets, because only these three algorithms can scale to such a big dataset. We use Faiss to build the KNN graph for the NSSG on the Deep100M dataset as the NSG does. It is because nn -descent does not work on our machine with such a large dataset. We test

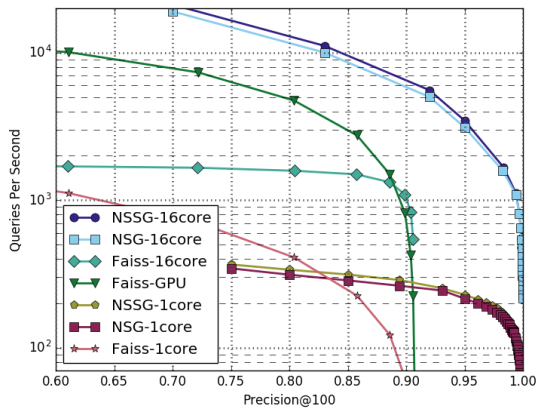


Figure 9: The results on Deep100M.

the inner-query parallel search performance on this dataset. The inner-query parallel search means conducting ANNS for one query with multiple threads. The NSG breaks the graph into 16 subsets randomly, builds 16 graphs on each subset, and searches for the results on these graphs in parallel [15]. Afterwards, the results from different working threads are merged to produce the final results. We follow the same routine to test the NSSG. The results are shown in Figure 9. “16core” means we test the algorithms with 16 threads for the inner-query parallel search. “1core” means we test the algorithm with only one thread, *i.e.*, we build a single index on the whole dataset.

We can see that the NSSG outperforms the NSG and Faiss on all settings. The improvement of the NSSG over the NSG is 5% to 15%. The NSSG can even outperform the Faiss-GPU version significantly. Moreover, the indexing time of the NSSG is much less than the NSG's. In [15], they take 3.53 hours to build the NSG-16core index, whereas we spend only 2.6 hours to build the NSSG-16core index. It takes us 11 hours to build the NSSG-1core index, 5 hours less than the NSG-1core's. This result further proves that the indexing complexity of the NSSG is much lower than the NSG's. We further optimize Alg. 2 for large datasets. Specifically, we do not need to keep the KNN graph in memory. We calculate the K nearest neighbors for each node on-the-fly instead. Therefore, we only use 56 GB memory at most, which is 37 GB less than the NSG. However, the NSG must keep the KNN graph in the memory because the key part of the NSG indexing is to search on the KNN graph and collect candidate neighbors for each node [15]. From the experiment results above, we can conclude that the NSSG is more scalable than the NSG in large applications.

7. CONCLUSION

In this paper, we propose a novel graph structure named as Satellite System Graph, which has superior properties for the ANNS problem. We provide detailed theoretical analysis for the SSG family and propose a practical variant, the Navigating SSG, for large-scale problems. Extensive experiments on medium and large-scale datasets have demonstrated the significant advantages of the NSSG over the other state-of-the-art approaches and also verified our theoretical analysis. Compared with the recent best-performing algorithm NSG,

the NSSG has lower searching complexity, lower indexing complexity. In summary, the NSSG have better scalability than the NSG. Our code has been released on GitHub¹⁵.

APPENDIX

A. THEORETICAL ANALYSIS

A.1 Proof For Theorem 1

Proof. According to the Lemma 1 in the NSG paper [15], in an Euclidean space E^d , a graph G defined on a finite point set S is monotonic if and only if for any two unconnected points $p, q \in S$, there is at least one edge $\overrightarrow{pr}$ such that $r \in B(q, \delta(p, q))$. $B(q, \delta(p, q))$ is defined as the open sphere centered at q with radius $\delta(p, q)$.

For any two node $p, q \in S$, there are only two kinds of relationships between them according to the definition of SSG.

- 1) p, q is connected. If p, q is connected, the path between p, q contains only one edge, which is monotonic.
- 2) p, q is not connected. If p, q is not connected, there must be at least one edge $\overrightarrow{pr} \in SSG$ such that $\overrightarrow{pr} \in Cone(\overrightarrow{pq}, \alpha)$ and $\delta(p, r) < \delta(p, q)$. Because $\angle rpq < \alpha \leq 60^\circ$, rq is not the longest edge in $\triangle pqr$, and we have $\delta(r, q) < \delta(p, q)$. Thus it is monotonic towards q when going from p to r . This process can be repeated iteratively until we have a monotonic search path from p to q . Finally, we have SSG is monotonic.

Since SSG is monotonic, according to the definition of Monotonic Search Network [15], SSG is strongly connected.

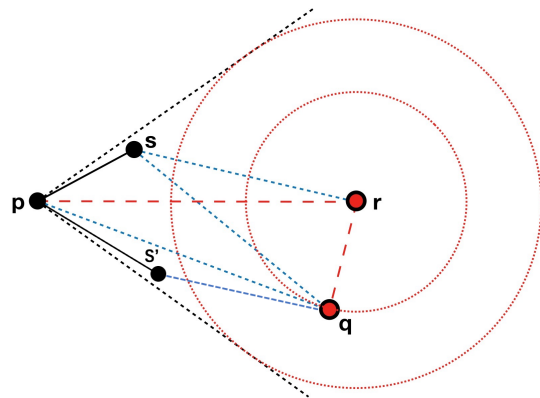


Figure 10: An illustration of the proof for the partially monotone path towards the query which is not in the database. Note that p, q, r, s may not be in the same hyper-plane.

A.2 Proof For Theorem 2

First we need to prove that the upper-bound of SSG's degree is a constant.

Lemma 1. *In an Euclidean space E^d , for any $0 < \alpha < \pi$, there are finite edges sharing a common end node such that the angle between any two of them is β and $\beta > \alpha$. The*

¹⁵<https://github.com/ZJULearning/SSG>

supremum of the number of such edges is a function of α and d .

Proof. Yao[38] prove that, for any $0 < \alpha < \pi$, one can cover the space E^d with finite convex cones such that $\sup\{\Theta(C_i)|C_i \in \mathbb{C}\} = \varphi$. Let $\mathbb{C} = \{C_1, C_2, \dots, C_k\}$ be a set of convex cones constructed by the algorithm proposed by Yao and with $\varphi = \frac{1}{2}\alpha - \varepsilon$, where ε is a very small value and $\varepsilon > 0$. Let u be the center node and uv, uw be any two edges. We define two convex cones C_i, C_j to be *adjacent* if $\exists ua, ua \in (C_i \cap C_j)$.

We will first prove that the closest two edges (two edges forming the smallest angle) must be inside the same cone or two adjacent cones. Let $\angle abc$ be the angle between ba and bc . If uv and uw are in two cones, C_i and C_j , that are not adjacent, i.e., $C_i \cap C_j = \emptyset$, then $\inf\{\angle vuw|uv \in C_i, uw \in C_j\} = \gamma > 0$. If uv and uw are in two adjacent cones, it's obvious that $\inf\{\angle vuw\} = 0$. Thus, the closest two edges must appear in the same cone or two adjacent cones.

If uv and uw are inside of the same cone, then $\angle vuw \leq \varphi < \alpha$ because $\sup\{\Theta(C_i)|C_i \in \mathbb{C}\} = \varphi$. Suppose uv and uw are inside of any two *adjacent* convex cones. Let uz be an edge, $uz \in (C_i \cap C_j)$. Because $\angle vuz \leq \varphi$ and $\angle wuz \leq \varphi$, uv and uw is inside the circular cone $\tilde{C}(\varphi, \mathbf{uz})$, and $\sup\{\angle vuw\} = 2\varphi = \alpha - 2\varepsilon < \alpha$. Suppose we select any k edges sharing a common end node and place them in $\mathbb{C}$. Given that $|\mathbb{C}| = k$, if there exists one empty cone, there must be two edges within the same cone. If we place one edge in each cone, the angle between any two edges is smaller than α . Therefore, there are at most m edges sharing a common end node such that the angle between any two of them is β , where $\beta \geq \alpha$ and $m < k$. m is a function of d and α , because they are the only variables involved. $\square$

Lemma 1 indicates that when dimension and angle are fixed, the degree will be upper-bounded by a constant. Then we can get the search complexity on SSG for in-database queries.

Proof. According to the Theorem 2 in [15], the length of the search paths for in-database queries is $O(n^{1/d} \log(n^{1/d})/\Delta r)$, where D is the degree upper-bound of SSG, n is the size of the dataset, and Δr is the lower-bound of lengths of edges in any non-isosceles triangles. Suppose D is the upper-bound of the degree of SSG, we can get the search complexity for in-database queries, $O(Dn^{1/d} \log(n^{1/d})/\Delta r)$, according to the Theorem 1 in [15]. $\square$

A.3 Proof For Theorem 3

Proof. Figure 10 is an illustration of this proof. At the initial stage of search, the search-starting point is far from the query and its nearest neighbor. Suppose q is the query which is not in the database, and r is q 's nearest neighbor which is within the database. According to the definition of SSG, we have at least one edge $\vec{ps}$ such that $\vec{ps} \in \text{Cone}(\vec{pr}, \alpha) \cap B(p, \delta(p, r)) \cap S$. Suppose plane Φ is the normal plane of the hyper-plane determined by point q, p, r and $pr \in \Phi$. We can get plane Φ divide $\text{Cone}(\vec{pr}, \alpha)$ into two parts evenly. Because the data are distributed randomly in the space, the probability of s to be within each half of the cone is 0.5.

1) If s is within the same half as q (like the position of s'), we can have $\angle ps'q < \alpha \leq 60^\circ$, and sq is not the longest edge in $\triangle pqs'$. Because $\forall ps' \in \text{SSG}, \delta(p, s') < \delta(p, q)$, we have

pq is the longest edge in $\triangle pqs'$, $\delta(p, q) > \delta(s'q)$. According to the definition of monotonicity [15], the search process is monotonic towards q . Because s' is also on the monotonic path to r according to the property of SSG, the search process is monotonic towards both q and r .

2) If s is within the other half, the search process is monotonic towards both q and r if $\angle spq < 60^\circ$, i.e., $\angle spr + \angle qpr < 60^\circ$. The probability of such an event is ϵ , $0 < \epsilon \leq 0.5$. So the total probability that the search process is monotonic towards both q and r is $0.5 + \epsilon$. It is easy for one to verify that if $\alpha < 30^\circ$, $\angle spq < 60^\circ$ definitely, and $\epsilon = 1$. $\square$

B. PARAMETERS

In our experiments, we gird-search the optimal parameters of the compared algorithms and NSSG. Below we list the values of some main parameters, which have big influences on the performance.

1. **SIFT1M.** We use $l = 100, r = 50, \alpha = 60^\circ, m = 10$ for NSSG, $l = 40, r = 50$ for NSG, $M = 25, efconstruction = 600$ for HNSW, $K = 400, L = 200, S = 10, R = 100, I = 20$ for KGraph, $L_1 = 500, S = 20, K = 400, L_2 = 200$ for DPG, $nlists = 4096$ for Faiss, $n_trees = 400$ for Annoy and *autotuned* algorithm for FLANN.
2. **GIST1M.** We use $l = 500, r = 70, \alpha = 60^\circ, m = 10$ for NSSG, $l = 60, r = 70$ for NSG, $M = 35, efconstruction = 800$ for HNSW, $K = 400, L = 400, R = 100, S = 15, I = 12$ for KGraph, $L_1 = 500, S = 20, K = 400, L_2 = 200$ for DPG, $nlists = 4096$ for Faiss, $n_trees = 400$ for Annoy and *autotuned* algorithm for FLANN.
3. **Crawl.** We use $l = 500, r = 40, \alpha = 60^\circ, m = 10$ for NSSG, $l = 500, r = 40$ for NSG, $M = 20, efconstruction = 1000$ for HNSW, $K = 400, L = 420, S = 15, R = 100, I = 12$ for KGraph, $L_1 = 500, S = 20, K = 400, L_2 = 200$ for DPG, $nlists = 4096$ for Faiss, $n_trees = 400$ for Annoy and *autotuned* algorithm for FLANN.
4. **GloVe.** We use $l = 500, r = 50, \alpha = 60^\circ, m = 10$ for NSSG, $l = 150, r = 50$ for NSG, $M = 25, efconstruction = 2500$ for HNSW, $K = 400, R = 420, S = 20, R = 200, I = 12$ for KGraph, $L_1 = 500, S = 20, K = 400, L_2 = 200$ for DPG, $nlists = 4096$ for Faiss, $n_trees = 500$ for Annoy and *autotuned* algorithm for FLANN.

C. REFERENCES

- [1] A. Arora, S. Sinha, P. Kumar, and A. Bhattacharya. Hd-index: Pushing the scalability-accuracy boundary for approximate knn search in high-dimensional spaces. *PVLDB*, 11(8):906–919, 2018.
- [2] S. Arya and D. M. Mount. Approximate nearest neighbor queries in fixed dimensions. In *SODA*, volume 93, pages 271–280, 1993.
- [3] F. Aurenhammer. Voronoi diagrams: a survey of a fundamental geometric data structure. *ACM Computing Surveys (CSUR)*, 23(3):345–405, 1991.
- [4] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger. The R*-tree: an efficient and robust access method for points and rectangles. In *ACM Sigmod Record*, volume 19, pages 322–331. Acm, 1990.
- [5] J. S. Beis and D. G. Lowe. Shape indexing using approximate nearest-neighbour search in high-dimensional spaces. In *1997 Conference on Computer Vision and Pattern Recognition*, pages 1000–1006, 1997.
- [6] H. Ben and D. Tom. FANNG: Fast approximate nearest neighbour graphs. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition*, pages 5713–5722, 2016.
- [7] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [8] M. Boguna, D. Krioukov, and K. C. Claffy. Navigability of complex networks. *Nature Physics*, 5(1):74–80, 2009.
- [9] L. Chen, M. T. Özsu, and V. Oria. Robust and fast similarity search for moving object trajectories. In *Proceedings of the 2005 ACM SIGMOD*, pages 491–502. ACM, 2005.
- [10] D. Dearholt, N. Gonzales, and G. Kurup. Monotonic search networks for computer vision databases. In *Signals, Systems and Computers, 1988.*, volume 2, pages 548–553. IEEE, 1988.
- [11] W. Dong, C. Moses, and K. Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web*, pages 577–586, 2011.
- [12] H. Ferhatosmanoglu, E. Tuncel, D. Agrawal, and A. El Abbadi. Approximate nearest neighbor searching in multimedia databases. In *Data Engineering, 2001.*, pages 503–511. IEEE, 2001.
- [13] A. W. Fu, P. M. Chan, Y. L. Cheung, and Y. S. Moon. Dynamic vp-tree indexing for n-nearest neighbor search given pair-wise distances. *Vldb Journal*, 9(2):154–173, 2000.
- [14] C. Fu and D. Cai. Efanna : An extremely fast approximate nearest neighbor search algorithm based on knn graph. *arXiv:1609.07228*, 2016.
- [15] C. Fu, C. Xiang, C. Wang, and D. Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB*, 12(5):461–474, 2019.
- [16] K. Fukunaga and P. M. Narendra. A branch and bound algorithm for computing k-nearest neighbors. *IEEE Transactions on Computers*, 100(7):750–753, 1975.
- [17] T. Ge, K. He, Q. Ke, and J. Sun. Optimized product quantization for approximate nearest neighbor search. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2946–2953, 2013.
- [18] T. Ge, K. He, Q. Ke, and J. Sun. Optimized product quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(4):744–755, 2014.
- [19] A. Gionis, P. Indyk, and R. Motwani. Similarity search in high dimensions via hashing. In *PVLDB*, pages 518–529, 1999.
- [20] K. Hajebi, Y. Abbasi-Yadkori, H. Shahbazi, and H. Zhang. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *IJCAI 2011*, volume 22, pages 1312–1317, 2011.
- [21] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *PVLDB*, 9(1):1–12, 2015.
- [22] Q. Huang, J. Feng, Y. Zhang, Q. Fang, and W. Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *PVLDB*, 9(1):1–12, 2015.
- [23] H. V. Jagadish, B. C. Ooi, K. L. Tan, C. Yu, and R. Zhang. idistance: An adaptive b + -tree based indexing method for nearest neighbor search. *ACM Transactions on Database Systems*, 30(2):364–397, 2005.
- [24] M. D. Jegou, Herve and C. Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [25] Z. Jin, D. Zhang, Y. Hu, S. Lin, D. Cai, and X. He. Fast and accurate hashing via iterative nearest neighbors expansion. *IEEE Transactions on Cybernetics*, 44(11):2167–2177, 2014.
- [26] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with gpus. *arXiv:1702.08734*, 2017.
- [27] J. M. Kleinberg. Navigation in a small world. *Nature*, 406(6798):845–845, 2000.
- [28] D.-T. Lee and B. J. Schachter. Two algorithms for constructing a delaunay triangulation. *International Journal of Computer & Information Sciences*, 9(3):219–242, 1980.
- [29] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin. Approximate nearest neighbor search on high dimensional data-experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering*, 2019.
- [30] T. Liu, C. R. Rosenberg, and H. A. Rowley. Clustering billions of images with large scale nearest neighbor search. In *8th IEEE Workshop on Applications of Computer Vision*, page 28, 2007.
- [31] X. Liu, C. Deng, B. Lang, D. Tao, and X. Li. Query-adaptive reciprocal hash tables for nearest neighbor search. *IEEE Transactions on Image Processing*, 25(2):907–919, 2016.
- [32] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems*, 45:61–68, 2014.
- [33] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov. Approximate nearest neighbor algorithm

- based on navigable small world graphs. *Inf. Syst.*, 45:61–68, 2014.
- [34] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
 - [35] J. Philbin, O. Chum, M. Isard, J. Sivic, and A. Zisserman. Object retrieval with large vocabularies and fast spatial matching. In *Computer Vision and Pattern Recognition, 2007. CVPR'07. IEEE Conference on*, pages 1–8. IEEE, 2007.
 - [36] C. Silpa-Anan and R. Hartley. Optimised kd-trees for fast image descriptor matching. In *Proceedings of the 2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2008.
 - [37] Y. Weiss, A. Torralba, and R. Fergus. Spectral hashing. In *Advances in Neural Information Processing Systems*, pages 1753–1760, 2009.
 - [38] A. C.-C. Yao. On constructing minimum spanning trees in k-dimensional spaces and related problems. *SIAM Journal on Computing*, 11(4):721–736, 1982.
 - [39] T. Zhang, C. Du, and J. Wang. Composite quantization for approximate nearest neighbor search. In *ICML*, number 2, pages 838–846, 2014.
 - [40] Y. Zheng, Q. Guo, A. K. Tung, and S. Wu. Lazyish: Approximate nearest neighbor search for multiple distance functions with a single index. *Proceedings of the 2016 International Conference on Management of Data*, pages 2023–2037, 2016.